

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS

COURSE CODE: CSA0706

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools.
(BL4,BL5)

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I **M Nishanth (192511154)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Nishanth

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and

stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Annexure A — Pseudocode Writing Task (Algorithm Design)

Question 1 — Throughput & Packet Loss Calculator

Given: total data received = 8000 bits, time = 4 seconds, packets sent = 200, packets received = 180. Calculate throughput (bps) and packet loss (%), validate inputs, classify the network, and display a full summary.

1. Problem Understanding

- **Inputs:** total_bits_received, time_seconds, packets_sent, packets_received.
- **Outputs:** throughput (bps), packet loss (%), and an Efficient/Inefficient classification with a summary.
- **Constraints:** time_seconds and packets_sent must not be zero (would cause division-by-zero); Efficient requires throughput > 1500 bps AND packet loss < 10%.

2. Pseudocode

```
BEGIN NetworkThroughputAndPacketLoss

// ---- Inputs ----
INPUT total_bits_received      // e.g. 8000
INPUT time_seconds             // e.g. 4
INPUT packets_sent             // e.g. 200
INPUT packets_received         // e.g. 180

// ---- Validation ----
IF time_seconds <= 0 THEN
    DISPLAY "Error: Time must be greater than zero."
    EXIT ALGORITHM
END IF

IF packets_sent <= 0 THEN
    DISPLAY "Error: Total packets sent must be greater than zero."
    EXIT ALGORITHM
END IF

// ---- Function: Throughput ----
FUNCTION CalculateThroughput(total_bits_received, time_seconds)
    throughput <- total_bits_received / time_seconds    // bits per second
    RETURN throughput
```

```

END FUNCTION

// ---- Function: Packet Loss ----
FUNCTION CalculatePacketLoss(packets_sent, packets_received)
    lost_packets <- packets_sent - packets_received
    loss_percentage <- (lost_packets / packets_sent) * 100
    RETURN loss_percentage
END FUNCTION

throughput <- CalculateThroughput(total_bits_received, time_seconds)
loss_percentage <- CalculatePacketLoss(packets_sent, packets_received)

// ---- Classification ----
IF throughput > 1500 AND loss_percentage < 10 THEN
    status <- "Efficient"
ELSE
    status <- "Inefficient"
END IF

// ---- Summary Output ----
DISPLAY "----- Network Performance Summary -----"
DISPLAY "Throughput      : " + throughput + " bps"
DISPLAY "Packet Loss     : " + loss_percentage + " %"
DISPLAY "Network Status : " + status
DISPLAY "-----"

END

```

3. Trace / Verification

Trace with the given values (8000 bits, 4 s, 200 sent, 180 received):

$\text{throughput} = 8000 / 4 = 2000 \text{ bps}$ ($> 1500 \rightarrow$ passes throughput check)

$\text{loss_percentage} = (200-180)/200 * 100 = 10\%$ (NOT $< 10 \rightarrow$ fails loss check)

Result: status = "Inefficient" (fails only on the packet-loss condition).

Question 2 — Bandwidth Utilization Monitoring & Backup Switching

Monitor bandwidth usage of multiple links continuously, store values, detect links exceeding 80% utilization, and recommend switching to a backup link.

1. Problem Understanding

- **Inputs:** a list of network links, each link's bandwidth capacity, its current usage (read periodically), and whether it has a backup link.

- **Outputs:** a running utilization log per link, warnings for overloaded links, and a switch-to-backup action/recommendation.
- **Constraints:** monitoring must be continuous (a loop, not a one-time check), and the overload threshold is fixed at 80%.

2. Pseudocode

```
BEGIN BandwidthMonitoring

// ---- Setup ----
DECLARE link_names[]           // list of monitored links, e.g. [L1, L2, L3, ...]
DECLARE bandwidth_capacity[]   // max capacity (Mbps) of each link
DECLARE has_backup[]           // TRUE/FALSE per link
DECLARE utilization_log[][]     // history of utilization readings per link
monitoring_interval <- 60      // seconds between checks

// ---- Function: read current usage from monitoring tool/SNMP ----
FUNCTION GetCurrentUsage(link)
    RETURN current_usage_of(link)
END FUNCTION

// ---- Function: convert usage to a percentage ----
FUNCTION CalculateUtilization(current_usage, capacity)
    utilization <- (current_usage / capacity) * 100
    RETURN utilization
END FUNCTION

// ---- Continuous Monitoring Loop ----
WHILE network_is_active DO

    FOR EACH link IN link_names DO

        current_usage <- GetCurrentUsage(link)
        utilization <- CalculateUtilization(current_usage, bandwidth_capacity[link])
        APPEND utilization TO utilization_log[link]      // store for trend analysis

        DISPLAY "Link: " + link + " | Utilization: " + utilization + "%"

        // ---- Overload Detection ----
        IF utilization > 80 THEN
            DISPLAY "WARNING: " + link + " is overloaded (" + utilization + "%)"

            IF has_backup[link] = TRUE THEN
                SwitchTraffic(link, BackupLinkOf(link))
                DISPLAY "Traffic redirected from " + link + " to backup link."
            ELSE
                DISPLAY "ALERT: No backup available for " + link + ". Notify
administrator."
```

```
        END IF
    END IF

    END FOR

    WAIT(monitoring_interval)    // pause before next monitoring cycle

    END WHILE

END
```

3. How This Supports Efficient Bandwidth Management

- Continuous monitoring (the outer WHILE loop) means overload is detected within one monitoring interval instead of only when users notice slowness.
- Storing each reading in `utilization_log` allows trend analysis — the administrator can see if a link is gradually trending toward overload and act pre-emptively.
- The 80% threshold acts as an early-warning point, triggering failover before a link is fully saturated, which prevents queuing delay and packet drops that occur near 100% utilization.
- Automatic redirection to a backup link (when available) keeps traffic flowing without waiting for manual administrator intervention, directly preventing congestion-related downtime.
- Where no backup exists, the algorithm still raises an alert, ensuring the administrator is informed and can provision capacity or a backup link proactively.

Question 3 — Best-Path Selection Using Link Quality Score

Six branch offices connect to headquarters over multiple links with changing bandwidth, delay, and utilization. Continuously identify the best path per branch using an overall quality score, and fail over automatically when a link is overloaded or down.

1. Problem Understanding

- **Inputs:** per link — bandwidth, delay, utilization, and reachability status; collected periodically for every candidate link of every branch.
- **Outputs:** the selected best link per branch, path-switch notifications, and alerts for overloaded/failed links.

- **Constraints:** selection must use a combined quality score (not distance alone); a DOWN or >90% utilized link must never be selected; monitoring repeats at regular intervals to adapt to changing conditions.

2. Pseudocode

```
BEGIN BestPathSelection

// ---- Setup ----
DECLARE branches[]           // 6 branch offices
DECLARE links[branch]       // candidate links for each branch
DECLARE bandwidth[link], delay[link], utilization[link]
DECLARE link_status[link]   // UP / DOWN
DECLARE current_best_path[branch]

W_BW    <- 0.5    // weight: higher bandwidth is better
W_DELAY <- 0.3    // weight: lower delay is better
W_UTIL  <- 0.2    // weight: lower utilization is better
monitoring_interval <- 30    // seconds

// ---- Function: collect live metrics for one link ----
FUNCTION CollectMetrics(link)
    bandwidth[link] <- ReadBandwidth(link)
    delay[link]     <- ReadDelay(link)
    utilization[link] <- ReadUtilization(link)
    IF LinkIsReachable(link) THEN
        link_status[link] <- "UP"
    ELSE
        link_status[link] <- "DOWN"
    END IF
END FUNCTION

// ---- Function: composite link-quality score ----
FUNCTION CalculateLinkQuality(link)
    IF link_status[link] = "DOWN" THEN
        RETURN -INFINITY    // never select a failed link
    END IF
    score <- (W_BW * bandwidth[link]) - (W_DELAY * delay[link]) - (W_UTIL *
utilization[link])
    RETURN score
END FUNCTION

// ---- Continuous Monitoring Loop ----
WHILE network_is_active DO

    FOR EACH branch IN branches DO

        best_score <- -INFINITY
        best_link  <- NULL
```



```

FOR EACH link IN links[branch] DO

    CollectMetrics(link)

    // ---- Overload / Failure Detection ----
    IF link_status[link] = "DOWN" OR utilization[link] > 90 THEN
        DISPLAY "ALERT: Link " + link + " to " + branch + " is
overloaded/down."
        CONTINUE // skip this link when scoring
    END IF

    score <- CalculateLinkQuality(link)

    IF score > best_score THEN
        best_score <- score
        best_link <- link
    END IF

END FOR

// ---- Path Selection / Automatic Failover ----
IF best_link <> NULL THEN
    IF best_link <> current_best_path[branch] THEN
        DISPLAY "Switching path for " + branch + " to " + best_link
        current_best_path[branch] <- best_link
    END IF
ELSE
    DISPLAY "CRITICAL ALERT: No usable link available for " + branch
END IF

END FOR

WAIT(monitoring_interval)

END WHILE

END

```

3. How This Improves Reliability, Routing Efficiency, and Fault Tolerance

- Reliability: excluding DOWN or >90%-utilized links from scoring (the CONTINUE step) guarantees traffic is never routed onto a link that is failing or saturated.
- Routing efficiency: the weighted score (W_BW, W_DELAY, W_UTIL) chooses the path with the best overall quality rather than the shortest hop count, so a short but congested or high-delay path is correctly avoided in favor of a better-performing longer path.

- Fault tolerance: because metrics are re-collected every monitoring_interval inside the WHILE loop, a link that fails mid-day is detected on the very next cycle and the algorithm automatically fails over to the next-best-scoring link without manual reconfiguration.
- Adaptability: recalculating scores every cycle means the chosen path continuously reflects current conditions (e.g., a link that becomes congested during peak hours is automatically deprioritized and traffic shifts to a fresher link).
- Administrator awareness: DISPLAY/ALERT statements for overloaded links, failures, and path switches keep the administrator informed in real time, supporting faster manual intervention where needed.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized	Handles most cases; reasonable	Handles basic cases only	Incomplete or inefficient
		approach	efficiency		